

Conception et Développement d'un Système Intelligent de Tracking (Architecture Microservices & IA Locale)

TECH - 57

Département Développement & Recherche

Plan de Développement Équitable

Stack : Spring Boot (Microservices) | Angular | PostgreSQL/PostGIS |
Python (Ollama/IA)

Version 2.0 – 16 Juillet 2026

Date de livraison impérative : 30 Août 2026

Table des matières

1	Présentation Générale et Contexte	3
1.1	Contraintes Temporelles	3
1.2	Philosophie de l'Équipe	3
2	Architecture Technique (Microservices)	4
2.1	Découpage en Microservices	4
2.2	Base de Données	4
3	Spécifications Détaillées des Interfaces Utilisateur	5
3.1	Interface d'Authentification (Page de Connexion)	5
3.2	Interface Administrateur (Pilotage global)	5
3.2.1	Dashboard Administrateur	5
3.2.2	Gestion des Comptes Utilisateurs	6
3.2.3	Gestion du Référentiel (Établissements)	6
3.2.4	Validation des Rapports	6
3.2.5	Paramètres Système	6
3.3	Interface Technicien (Cœur Opérationnel)	6
3.3.1	Dashboard Technicien	6
3.3.2	Planification des Missions	7
3.3.3	Wizard de calcul du meilleur trajet (Fonctionnalité prioritaire)	7
3.3.4	Exécution de l'Intervention (Check-in / Check-out)	7
3.3.5	Gestion des Matériels et Checklist	8
3.3.6	Signature Électronique et Clôture	8
3.4	Interface Observateur / Client (Lecture seule)	8
3.4.1	Dashboard Client	8
3.4.2	Espace Documents (Impression)	8
3.4.3	Ressources Pédagogiques (Module Client)	8
3.4.4	Assistant IA (Chatbot)	9
3.5	Composants Transversaux Communs	9
4	Répartition Équitable des Tâches (3 Devs)	10
4.1	Principe de l'Équité	10
4.2	Partage des Compétences (Équité Active)	11
5	Planning Opérationnel (16 Juillet → 30 Août)	12
5.1	Stratégie de Sprints (6 Sprints de 1 semaine)	12

6	Stratégie DevOps et Qualité	15
6.1	Outilillage	15
6.2	Stratégie de Test (Responsabilité collective)	15
7	Spécifications IA (Python/Ollama)	16
8	Indicateurs et Livrables Finaux	17
8.1	KPIs de Suivi de Projet	17
8.2	Livrables Attendus le 30 Août	17
	Conclusion	18

Chapitre 1

Présentation Générale et Contexte

TECH-57 développe une plateforme de suivi des installations logistiques dans les établissements publics. Le projet intègre des microservices pour la scalabilité, un moteur IA local (Ollama/Python) pour la génération de rapports et l'assistance, ainsi qu'une chaîne DevOps complète.

1.1 Contraintes Temporelles

Début : 16 Juillet 2026

Livraison finale (production) : 30 Août 2026

Durée totale : 6,5 semaines (45 jours ouvrés).

1.2 Philosophie de l'Équipe

L'équipe est composée de 3 développeurs full-stack. Pour garantir l'**équité** et l'**engagement**, chaque membre est *lead* d'un domaine métier, mais tous participent aux couches transverses :

- **DevOps** (Docker, CI/CD, Monitoring).
- **IA** (Python, Ollama, RAG).
- **QA/Testing** (Tests unitaires, intégration, E2E, performance).

Aucun développeur n'est cantonné à un rôle "ennuyeux" ; la répartition est horizontale et collaborative.

Chapitre 2

Architecture Technique (Microservices)

2.1 Découpage en Microservices

- **API Gateway** : Spring Cloud Gateway (routage, authentification JWT).
- **Service Auth & Users** : Spring Boot (Gestion des comptes, rôles).
- **Service Référentiel** : Spring Boot (Régions, Provinces, Établissements, PostGIS).
- **Service Missions & Interventions** : Spring Boot (Check-in/out, états, photos).
- **Service Matériels & Inventaire** : Spring Boot (QR Code, mouvements, stock).
- **Service IA (Python)** : FastAPI + Ollama (modèle LLM local). Gère la génération de rapports et le chatbot RAG.
- **Service Reporting** : Spring Boot (Génération PDF/Excel).

2.2 Base de Données

PostgreSQL 15 avec extension PostGIS (calculs géospatiaux pour les distances et le meilleur trajet).

Chapitre 3

Spécifications Détaillées des Interfaces Utilisateur

Ce chapitre décrit précisément l'ensemble des écrans (UI/UX) de l'application. Chaque interface est définie par ses composants, ses données, ses règles d'affichage et les interactions autorisées selon le profil connecté.

3.1 Interface d'Authentification (Page de Connexion)

- **Composants** : Champs "Email / Identifiant", "Mot de passe", case à cocher "Se souvenir de moi", bouton "Se connecter", lien "Mot de passe oublié".
- **Règles** :
 1. Validation des champs (email valide, mot de passe non vide).
 2. Appel à l'API Gateway qui délègue au Service Auth.
 3. Réception du JWT (Access + Refresh Token). Stockage sécurisé dans le local-Storage / HttpOnly cookie.
 4. Redirection automatique vers le Dashboard du rôle associé (Admin, Tech ou Observateur).
- **Pages secondaires** : Formulaire de réinitialisation du mot de passe (envoi d'email).

3.2 Interface Administrateur (Pilotage global)

3.2.1 Dashboard Administrateur

- **Vue d'ensemble** : Cartographie interactive (Leaflet) positionnant tous les établissements avec des marqueurs colorés (Vert = OK, Orange = En cours, Rouge = Anomalie / Retard).
- **KPIs en temps réel** :
 - Nombre total d'établissements installés.
 - Taux de conformité global (%).
 - Nombre d'interventions en retard.
 - Temps moyen d'installation (sur les 7 derniers jours).
- **Widgets cliquables** : Chaque KPI est un bouton qui filtre la liste des interventions juste en dessous.

3.2.2 Gestion des Comptes Utilisateurs

- **Écran** : Tableau paginé listant tous les utilisateurs (Nom, Email, Rôle, Établissement rattaché, Statut : Actif/Inactif).
- **Formulaire de création/modification** :
 1. Champs : Nom, Prénom, Email, Rôle (Admin / Technicien / Observateur), Établissement assigné (obligatoire pour Tech/Observateur).
 2. Génération automatique d'un mot de passe temporaire envoyé par email.
 3. Boutons : "Créer", "Mettre à jour", "Désactiver" (le "Supprimer" est un soft-delete).
- **Règle spécifique** : L'Admin peut basculer un Observateur en Technicien, mais doit confirmer cette action (impact sur les permissions).

3.2.3 Gestion du Référentiel (Établissements)

- **Arborescence** : Vue hiérarchique (Région → Province → Commune → Établissement). Possibilité d'ajouter un établissement à n'importe quel niveau.
- **Formulaire Établissement** :
 - Référence unique (auto-générée ou manuelle).
 - Désignation, Adresse, Coordonnées GPS (avec un mini-maps pour géolocalisation manuelle).
 - Nombre de bénéficiaires, Responsable, Téléphone, Email.
- **Import/Export** : Boutons "Exporter en Excel" et "Importer depuis Excel" pour le chargement de masse des établissements.

3.2.4 Validation des Rapports

- **Liste d'attente** : Tableau des rapports générés par les techniciens, avec statut "En attente de validation".
- **Aperçu** : Visionneuse PDF intégrée (iframe/embed) pour lire le rapport.
- **Action** : Bouton "Valider" (le rapport passe en statut "Publié" et devient accessible à l'Observateur) ou "Rejeter" (avec obligation de saisir un motif de rejet).

3.2.5 Paramètres Système

- **Prix du gasoil** : Champ numérique (MAD/Litre) avec historique des mises à jour.
- **Seuils d'anomalie** : Définition du nombre de jours maximum autorisés avant qu'une intervention ne soit considérée comme "en retard".
- **Configuration IA** : Sélection du modèle Ollama (ex : llama3, mistral) et température du prompt.

3.3 Interface Technicien (Cœur Opérationnel)

3.3.1 Dashboard Technicien

- **Filtrage automatique** : Seules les missions assignées au technicien connecté sont affichées.

- **Vue agenda** : Calendrier listant les interventions de la semaine. Couleurs : Bleu (À venir), Vert (En cours), Gris (Terminé).
- **Statistiques personnelles** : Nombre d'établissements qu'il gère, taux de respect de ses délais personnels, quantité de matériel installé ce mois-ci.

3.3.2 Planification des Missions

- **Liste des missions à venir** : Tableau avec colonnes (Date, Établissement, Type d'installation, Statut).
- **Bouton "Démarrer"** : Ouvre le **Wizard de trajet** (décrit ci-dessous). Le statut passe automatiquement à "En cours".

3.3.3 Wizard de calcul du meilleur trajet (Fonctionnalité prioritaire)

C'est l'écran le plus critique pour le Technicien. Il se déroule en 3 étapes claires :

1. Étape 1 - Point de départ :

- Le champ est pré-rempli avec la dernière position GPS connue du technicien (ou saisie manuelle d'une adresse).
- Affichage sur une mini-carte pour validation visuelle.

2. Étape 2 - Point d'arrivée :

- Sélection automatique de l'établissement assigné à la mission (mais modifiable).

3. Étape 3 - Comparaison et choix :

- L'application calcule et affiche deux cartes :
- **Option A (Autoroute)** : Distance (km), Temps estimé, Prix du péage, Consommation gasoil, Coût total.
- **Option B (Nationale)** : Distance (km), Temps estimé, Pas de péage, Consommation gasoil, Coût total.
- **Recommandation IA** : Mise en avant de la "Meilleure offre" (moins chère) et de l'option "La plus rapide".

4. Action finale : Le technicien sélectionne son itinéraire, puis clique sur "Valider et Démarrer la mission". Le système enregistre le choix (traçabilité des frais).

3.3.4 Exécution de l'Intervention (Check-in / Check-out)

- **Check-in** : Écran avec un gros bouton "Débuter l'intervention". L'action horodate la date/heure et enregistre la position GPS actuelle (via l'API Géolocalisation du navigateur).
- **Pendant l'intervention** : Accès à une checklist déroulante.
- **Check-out** : Bouton "Terminer l'intervention" qui horodate la fin et calcule automatiquement la durée totale.

3.3.5 Gestion des Matériels et Checklist

- **Liste des équipements** : Tableau avec colonnes (Nom, Quantité prévue, Quantité installée, État). Des cases à cocher permettent de valider l'installation.
- **Scan QR Code** : Un bouton active la caméra du smartphone/ordinateur. Le scanner (bibliothèque `zxing`) détecte le QR Code sur l'équipement, affiche ses informations et propose de mettre à jour son statut (ex : "Installé" ou "Défectueux").
- **Upload Photos** : Zone de drag & drop pour ajouter des photos "Avant" et "Après". Les images sont compressées automatiquement avant envoi au backend.

3.3.6 Signature Électronique et Clôture

- **Signature Pad** : Zone de dessin (canvas) sur laquelle le responsable de l'établissement appose sa signature à l'aide d'une souris ou d'un stylet.
- **Génération du brouillon** : Bouton "Générer le rapport" qui déclenche l'appel au microservice IA (Python/Ollama). Le rapport texte apparaît dans un éditeur (textArea) où le technicien peut le modifier manuellement avant de le soumettre à la validation de l'Admin.

3.4 Interface Observateur / Client (Lecture seule)

3.4.1 Dashboard Client

- **Filtrage strict** : L'Observateur ne voit que les données de son propre établissement (ou des établissements qui lui sont rattachés).
- **Timeline** : Chronologie verticale des interventions passées et à venir (Dates, statuts, techniciens assignés).
- **KPIs personnalisés** : Nombre d'équipements installés dans son établissement, taux d'avancement de la mission en cours.

3.4.2 Espace Documents (Impression)

- **Liste des rapports** : Seuls les rapports avec le statut "Publié" (validé par l'Admin) apparaissent.
- **Actions autorisées** :
 - **Bouton "Télécharger PDF"** : Télécharge le fichier sur le poste du client.
 - **Bouton "Imprimer"** : Lance l'impression directe via `window.print()`.
- **Attestation de réalisation** : Document officiel généré automatiquement, reprenant les photos "Après" et la checklist validée.

3.4.3 Ressources Pédagogiques (Module Client)

- **Guides PDF** : Visualisation et téléchargement des manuels d'installation fournis par **TECH-57**.
- **Vidéos explicatives** : Lecteur HTML5 intégré (ou embed YouTube/Vimeo) listant des tutoriels sur l'utilisation et la maintenance des équipements.

3.4.4 Assistant IA (Chatbot)

- **Fenêtre de chat flottante** (comme un assistant virtuel) accessible depuis n'importe quelle page.
- **Fonctionnalité RAG** : Le client pose une question en langage naturel (ex : "Quand est-ce que le câblage sera terminé?"). Le frontend envoie la requête au service Python, qui interroge la base de données, construit un prompt contextuel et répond via Ollama.
- **Contraintes** : Les réponses sont limitées aux données de l'établissement de l'Observateur (grâce au filtrage par JWT).

3.5 Composants Transversaux Communs

- **Barre de navigation** : Adaptative selon le rôle. L'Admin voit "Gestion des comptes", le Tech voit "Mes missions", l'Observateur voit "Mes documents".
- **Notifications** : Système de Toast/Alert pour les actions réussies (ex : "Check-in effectué", "Rapport envoyé pour validation").
- **Export global** : Dans les tableaux (Admin/Tech), boutons d'export CSV/Excel disponibles avec les colonnes visibles.

Chapitre 4

Répartition Équitable des Tâches (3 Devs)

4.1 Principe de l'Équité

Chaque développeur aura une **responsabilité primaire** sur un microservice, mais sera **co-responsable** des tâches transverses. Les tâches DevOps, IA et Tests sont réparties en *leadership tournant* pour que chacun monte en compétence.

TABLE 4.1 – Répartition équitale des responsabilités

Développeur	Lead Métier (Micro-services)	Responsabilités Transverses (DevOps/IA/QA)	Livrables Clés
DEV 1	<i>Auth, Gateway & Géolocalisation</i> - Spring Cloud Gateway. - Service Auth/Users (JWT, rôles). - Service Référentiel (PostGIS).	- DevOps Lead : Rédaction des Dockerfiles, docker-compose global, scripts de déploiement. - Configuration du pipeline CI/CD (GitHub Actions). - Tests de charge (K6) sur l'API Gateway.	- Infra 1-Click. - Pipeline CI/CD fonctionnelle. - Endpoints géolocalisation opérationnels.
DEV 2	<i>Missions & Interface IA</i> - Service Missions/Interventions. - Service IA (Python/-FastAPI + Ollama). - Intégration du chatbot et RAG.	- IA Lead : Fine-tuning des prompts, intégration du modèle local. - Tests d'intégration Backend/Frontend (Testcontainers). - Mise en place du monitoring des logs pour l'IA.	- API Python (Ollama) fonctionnelle. - Génération automatique des rapports. - Composant Chatbot Angular connecté.
DEV 3	<i>Matériels, Reporting & Frontend</i> - Service Matériels/QR Code. - Service Reporting (PDF/Excel). - Application Angular (tous les composants UI).	- QA Lead : Écriture des tests E2E (Cypress) et tests de sécurité (OWASP). - Intégration de SonarQube. - Gestion des environnements (dev/staging/prod) pour le Frontend.	- Scanner QR Code fonctionnel. - Export PDF/Excel. - Application Angular responsive et testée.

4.2 Partage des Compétences (Équité Active)

Pour éviter la frustration, des **binômes de compétences** sont prévus :

- **Semaine 1 & 2** : DEV 2 aide DEV 1 sur le déploiement Docker ; DEV 1 aide DEV 3 sur la sécurité des endpoints.
- **Semaine 3 & 4** : DEV 3 aide DEV 2 sur l'intégration de l'IA dans le frontend ; DEV 1 aide DEV 2 sur les appels API Python.
- **Semaine 5 & 6** : Tous les 3 se concentrent sur la **QA collective** (tests de non-régression et déploiement).

Chapitre 5

Planning Opérationnel (16 Juillet → 30 Août)

5.1 Stratégie de Sprints (6 Sprints de 1 semaine)

Le projet est découpé en 6 sprints d'une semaine, suivis de 3 jours de recette finale.

TABLE 5.1 – Planning détaillé sur 6,5 semaines

Période	Sprint	Objectifs Principaux (Tous les Devs)	Livraison
16-22 Juil	S0 (Setup)	- Installation de l'environnement (JDK, Python, Ollama, PostgreSQL/PostGIS). - Initialisation des projets (Spring Initializr, Angular CLI, FastAPI). - Rédaction des premiers Dockerfiles et du docker-compose.yml. - Modélisation de la base de données (Dev1 lead, Dev2&3 assistant).	Dépôt Git structuré, application qui se lance en Docker.
23-29 Juil	S1 (Auth & Référentiel)	- DEV1 : API Gateway + Auth JWT + CRUD Établissements (PostGIS). - DEV2 : Création du service Missions (entités de base). - DEV3 : Création du service Matériels + squelette Angular (layout, guards). - Transverse : Tous écrivent leurs tests unitaires (JUnit / PyTest).	Authentification fonctionnelle, CRUD de base opérationnels en local.

Période	Sprint	Objectifs	Livraison
30 Juil - 5 Août	S2 (Cœur Métier & IA PoC)	- DEV1 : Implémentation du calcul de distance (PostGIS) et endpoint "Meilleur Trajet". - DEV2 : Intégration d'Ollama en Python (premier test de génération de rapport). - DEV3 : Intégration du QR Code (zxing) dans Angular + API de scan. - Transverse : Mise en place de SonarQube dans la pipeline (Dev1 aide Dev3).	API de trajet opérationnelle, premier prompt IA réussi, scan QR.
6-12 Août	S3 (Expérience Terrain)	- DEV1 : Finalisation du wizard de trajet (comparaison Autoroute/Nationale). - DEV2 : Implémentation du Check-in/out, upload photos, signature électronique. - DEV3 : Construction des dashboards Angular (Admin, Tech, Client). - Transverse : Écriture des tests d'intégration (Testcontainers) pour les workflows critiques.	Parcours Technicien complet : connexion → trajet → check-in → fin.
13-19 Août	S4 (Reporting & IA Avancée)	- DEV1 : Génération PDF (iText) et endpoint de validation Admin. - DEV2 : Implémentation du RAG (enrichissement du prompt avec données BDD) + Chatbot. - DEV3 : Finalisation du module Client (téléchargement, vidéos, impression). - Transverse : Tests E2E (Cypress) sur l'ensemble du parcours (Dev3 lead, tous écrivent des scénarios).	Rapports IA générés, Chatbot fonctionnel, documents imprimables.
20-26 Août	S5 (QA, Performance & Staging)	- Tous les 3 en mode "Recette" : Nettoyage de la dette technique, correction des bugs. - Tests de charge (K6) sur les endpoints critiques (Dev1). - Durcissement de la sécurité (Dev3). - Déploiement sur un serveur de Staging (DevOps collectif).	Version Staging prête, couverture de tests > 80%, aucun bug critique.

Période	Sprint	Objectifs	Livraison
27-30 Août	S6 (Déploiement Final)	- Migration de la base de données en production. - Déploiement final sur le serveur de production (ou cloud). - Rédaction de la documentation technique et utilisateur. - Livraison finale au client le 30 Août à 18h00.	Application en production, documentation remise.

Chapitre 6

Stratégie DevOps et Qualité

6.1 Outilillage

- **CI/CD** : GitHub Actions (Build, Test, Security Scan, Push Docker).
- **Conteneurs** : Docker & Docker Compose. Chaque microservice possède son propre Dockerfile.
- **Monitoring** : Prometheus + Grafana (Dev1 lead, configuré en S4).
- **Qualité** : SonarQube (Dev3 lead) et OWASP Dependency Check.

6.2 Stratégie de Test (Responsabilité collective)

- **Tests Unitaires** : Chaque Dev écrit les tests pour ses propres services (JUnit 5 pour Spring, PyTest pour Python, Jasmine/Karma pour Angular).
- **Tests d'Intégration** : Testcontainers pour valider les interactions entre microservices (Dev2 lead).
- **Tests E2E** : Cypress couvrant les parcours Admin, Tech et Observateur (Dev3 lead).
- **Tests de Performance** : K6 pour simuler 200 techniciens en simultané (Dev1 lead).

Chapitre 7

Spécifications IA (Python/Ollama)

L'IA est un microservice à part entière :

- **Technologie** : FastAPI + Ollama (modèle Mistral ou Llama 3, quantifié pour tourner localement).
- **Fonctionnalités** :
 1. **Génération de rapport** : Le service reçoit un ID de mission. Il interroge la BDD (via REST), construit un prompt contextuel, interroge Ollama et retourne le texte formaté.
 2. **Chatbot RAG** : Les données des établissements/missions sont vectorisées (via un simple embeddings local) et utilisées pour enrichir les réponses.
- **Responsabilité** : DEV2 est le lead sur ce composant, mais DEV1 et DEV3 participent à la définition des prompts et à l'intégration dans le frontend.

Chapitre 8

Indicateurs et Livrables Finaux

8.1 KPIs de Suivi de Projet

- Vitesse de l'équipe (points par sprint).
- Couverture de code (minimum 80%).
- Temps de réponse moyen de l'API (< 500 ms en staging).

8.2 Livrables Attendus le 30 Août

1. Code source complet (Backend Spring, Python, Frontend Angular).
2. Scripts de déploiement (Docker, CI/CD).
3. Rapport de tests (unitaires, intégration, E2E, performance).
4. Documentation utilisateur (Guide Admin, Tech, Observateur).
5. Accès à l'application déployée en production.

Conclusion et Engagement

Cette répartition équitable assure que chaque développeur développe des compétences en **DevOps**, **IA** et **Testing**, tout en restant lead sur un domaine métier spécifique. Le planning serré (45 jours) est tenable grâce à la méthode Agile et à la forte cohésion d'équipe.

Nous nous engageons à livrer une plateforme robuste, scalable et intelligente d'ici le **30 Août 2026**.

Approuvé par l'Équipe Projet

Date : 16 Juillet 2026